

Android Developer Fundamentals V2

Activities et Intents

Lesson 2



2.2 Cycle de vie (lifecycle) et etat d'une ativite

Sommaire

- Cycle de vie d'une activite
- Callbacks du cycle de vie de l'activité
- État de l'instance d'activité
- Sauvegarde et restauration de l'état de l'activité

Cycle de vie d'une activite (lifecycle)

Qu'est-ce que le cycle de vie des activités ?

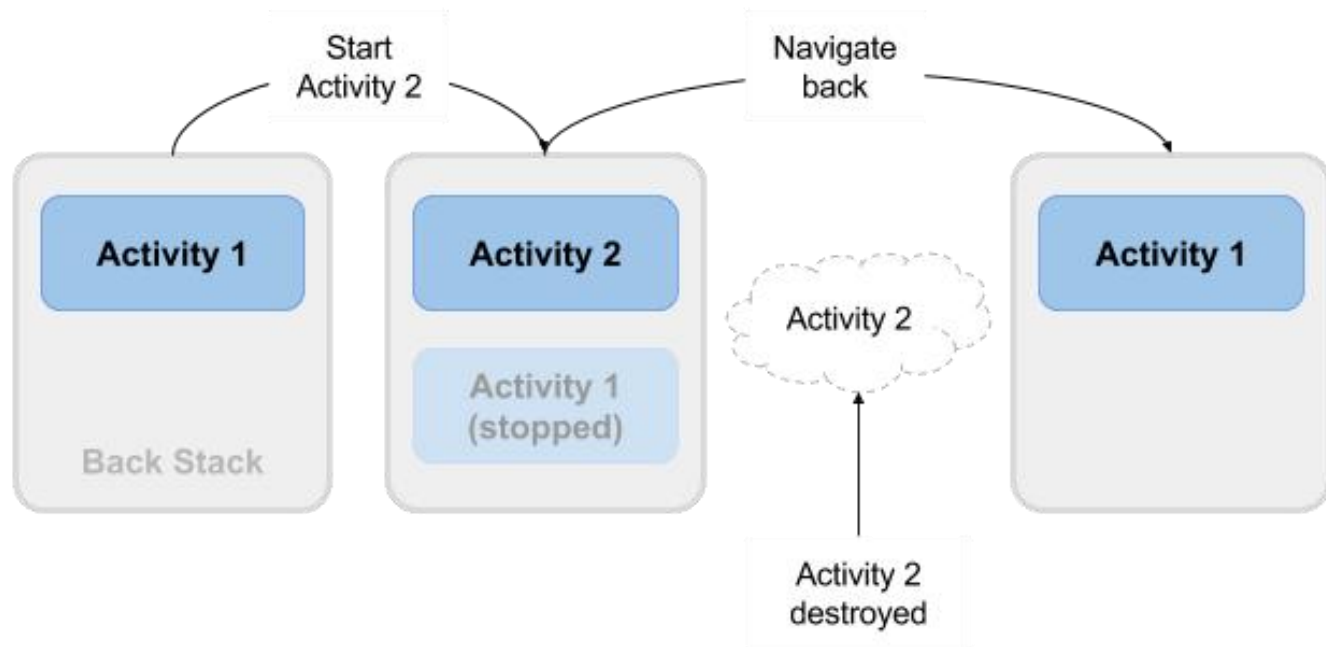
- L'ensemble des états dans lesquels une activité peut se trouver pendant sa durée de vie, depuis sa création jusqu'à sa destruction.

Plus formellement:

- Un graphe orienté de tous les états dans lesquels une activité peut se trouver, et les rappels associés à la transition de chaque état au suivant.



Qu'est-ce que le cycle de vie des activités ?



États d'activité et visibilité de l'application

- Created(pas encore visible)
- Started (visible)
- Resume(visible)
- Paused (partiellement invisible)
- Stopped (caché)
- Destroyed (disparu de la mémoire)

Les changements d'état sont déclenchés par une action de l'utilisateur, des modifications de configuration telles que la rotation de l'appareil, ou une action du système.

Callbacks du cycle de vie de l'activité

Callbacks et quand ils sont appelés

onCreate(Bundle savedInstanceState)—Initialisation statique

onStart()—lorsque l'activité (écran) devient visible

onRestart()—appelé si l'activité a été arrêtée (appel onStart())

onResume()—commencer à interagir avec l'utilisateur

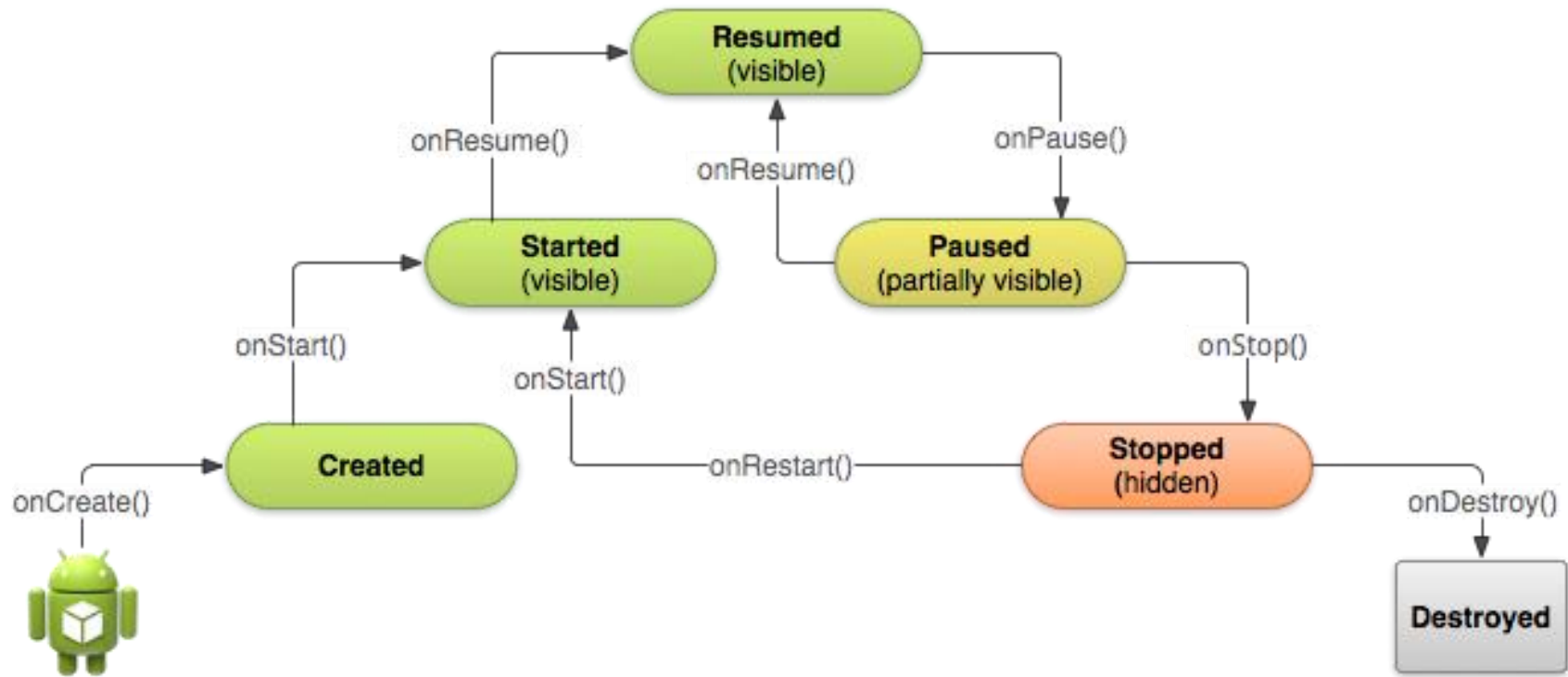
onPause()—sur le point de reprendre l'activité précédente

onStop()—n'est plus visible, mais existe toujours et toutes les informations relatives à l'état sont conservées

onDestroy()—dernier appel avant que le système Android ne détruise l'activité



Graphique des états d'activité et des rappels



Mise en œuvre et implementation des rappels

- Seul onCreate() est nécessaire
- Redéfinissez les autres méthodes de rappel (callbacks) pour modifier le comportement par défaut.

onCreate() → Created

- Appelée lorsque l'Activity est créée pour la première fois, par exemple lorsque l'utilisateur appuie sur l'icône du lanceur (launcher).
- Effectue toute la configuration statique : créer les vues, lier les données aux listes ...
- Appelée une seule fois pendant le cycle de vie d'une activité.
- Prend en paramètre un Bundle contenant l'état précédemment figé (sauvegardé) de l'activité, s'il existe.
- L'état créé est toujours suivi par l'appel de onStart().

onCreate(Bundle savedInstanceState)

@Override

```
public void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    // The activity is being created.  
}
```

onStart() → Started

- Appelée lorsque l'Activity devient visible pour l'utilisateur.
- Peut être appelée plusieurs fois durant le cycle de vie.
- Suivie par onResume() si l'activité passe au premier plan, ou par onStop() si elle devient cachée.

onStart()

```
@Override  
protected void onStart() {  
    super.onStart();  
    // The activity is about to become visible.  
}
```

onRestart() -> Started

- Appelée après que l'Activity a été arrêtée, juste avant qu'elle ne soit redémarrée.
- État transitoire
- Toujours suivi de onStart()

onRestart()

```
@Override  
protected void onRestart() {  
    super.onRestart();  
    // The activity is between stopped and started.  
}
```

onResume() → Resumed/Running

- Appelée lorsque l'Activity va commencer à interagir avec l'utilisateur.
- L'activité est passée au sommet de la pile d'activités.
- Commence à accepter les entrées de l'utilisateur.
- État de fonctionnement
- Toujours suivi de onPause()

onResume()

```
@Override  
protected void onResume() {  
    super.onResume();  
    // The activity has become visible  
    // it is now "resumed"  
}
```

onPause() → Paused

- Appelé lorsque le système est sur le point de reprendre une activité précédente
- L'activité est partiellement visible mais l'utilisateur quitte l'activité.
- Généralement utilisé pour valider les modifications non sauvegardées des données persistantes, arrêter les animations et tout ce qui consomme des ressources.
- Les implémentations doivent être rapides car l'activité suivante n'est pas reprise tant que cette méthode n'est pas renvoyée
- Suivi de onResume() si l'activité revient au premier plan, ou de onStop() si elle devient invisible pour l'utilisateur.

onPause()

@Override

```
protected void onPause() {  
    super.onPause();  
    // Another activity is taking focus  
    // this activity is about to be "paused"  
}
```

onStop() -> Stopped

- Appelé lorsque l'activité n'est plus visible par l'utilisateur
- Une nouvelle activité est démarrée, une activité existante est placée devant celle-ci, ou celle-ci est détruite
- Opérations qui étaient trop lourdes pour onPause()
- Suivies par onRestart() si l'activité revient pour interagir avec l'utilisateur, ou onDestroy() si l'activité s'en va.

onStop()

```
@Override  
protected void onStop() {  
    super.onStop();  
    // The activity is no longer visible  
    // it is now "stopped"  
}
```

onDestroy() → Destroyed

- Dernier appel avant la destruction de l'activité
- L'utilisateur revient à l'activité précédente ou la configuration change
- L'activité se termine ou le système la détruit pour économiser de l'espace
- Appelle la méthode `isFinishing()` pour vérifier
- Le système peut détruire l'activité sans appeler cette méthode, utilisez donc `onPause()` ou `onStop()` pour sauvegarder les données ou l'état.

onDestroy()

```
@Override  
protected void onDestroy() {  
    super.onDestroy();  
    // The activity is about to be destroyed.  
}
```

État de l'instance d'activité

Quand la configuration change-t-elle ?

Les changements de configuration invalident la présentation actuelle ou d'autres ressources de votre activité lorsque l'utilisateur:

- Fait pivoter l'appareil
- Choisit une autre langue système, ce qui entraîne un changement locale
- Entre en mode multifenêtre (à partir d'Android 7)

Que se passe-t-il en cas de changement de configuration ?

Lors d'un changement de configuration, Android:

1. Arrêt de l'activité en appelant:

- onPause()
- onStop()
- onDestroy()

2. Recommence l'activité en appelant:

- onCreate()
- onStart()
- onResume()

État de l'instance d'activité

- Des informations d'état sont créées pendant l'exécution de l'activité, telles qu'un compteur, un texte utilisateur, une progression de l'animation.
- L'état est perdu en cas de rotation de l'appareil, de changement de langue, d'appui sur la touche retour ou d'effacement de la mémoire.

Sauvegarde et restauration de l'état de l'activité

Ce que le système permet

- Sauvegarde du système uniquement:
 - État des vues avec un identifiant unique (android:id) tel que le texte saisi dans EditText
 - L'intent qui a déclenché l'activité et les données dans ses extras
- Vous êtes responsable de la sauvegarde des autres activités et des données relatives à la progression des utilisateurs.

Sauvegarde de l'état de l'instance

Implémenter `onSaveInstanceState()` dans votre activité

- Appelé par le runtime Android lorsqu'il y a une possibilité que l'activité soit détruite.
- Sauvegarde des données uniquement pour cette instance de l'activité pendant la session en cours

onSaveInstanceState(Bundle outState)

@Override

```
public void onSaveInstanceState(Bundle outState) {  
    super.onSaveInstanceState(outState);  
  
    // Add information for saving HelloToast counter  
    // to the to the outState bundle  
    outState.putString("count",  
        String.valueOf(mShowCount.getText()));  
}
```

Restauration de l'état de l'instance

Deux façons de récupérer le Bundle sauvegardé en

- `onCreate(Bundle mySavedState)` Préféré, pour s'assurer que l'interface utilisateur, y compris tout état sauvegardé, est de nouveau opérationnelle le plus rapidement possible
- Mise en œuvre d'un rappel (appelé après `onStart()`) `onRestoreInstanceState(Bundle mySavedState)`

Restauration dans onCreate()

@Override

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
    mShowCount = findViewById(R.id.show_count);  
  
    if (savedInstanceState != null) {  
        String count = savedInstanceState.getString("count");  
        if (mShowCount != null)  
            mShowCount.setText(count);  
    }  
}
```

onRestoreInstanceState(Bundle state)

```
@Override
public void onRestoreInstanceState (Bundle mySavedState) {
    super.onRestoreInstanceState(mySavedState);

    if (mySavedState != null) {
        String count = mySavedState.getString("count");
        if (count != null)
            mShowCount.setText(count);
    }
}
```

État de l'instance et redémarrage de

Lorsque vous arrêtez et redémarrez une nouvelle session d'application, les états de l'instance d'activité sont perdus et vos activités reprennent leur apparence par défaut.

Si vous devez sauvegarder les données de l'utilisateur entre les sessions de l'application, utilisez des préférences partagées ou une base de données.

FIN